컴퓨터구조 4주차 실습 노트 (LAB-04)

비트 연산 · 32비트 상수 · 곱셈과 나눗셈

1. 실습 개요와 준비

오늘은 세 가지 실습을 한다. 모든 실습은 "손으로 예측 → QtSpim으로 한 줄씩 실행 → 결과 비교" 순서로 진행하며, **실행 전에 기대값을 마지막 쪽 기록표에 먼저 적는다.**

실습	주제	명령어	핵심 질문
1	비트 연산 (Logical + Shift)	`and or xor nor sll srl sra`	마스크로 비트를 고르고, 시프트의 빈자리는 무엇으로 채우는가?
2	32비트 상수 만들기	`lui ori addi (li)`	immediate는 16비트뿐인데 32비트 상수는 어떻게 넣는가?
3	곱셈과 나눗셈	`mul mult div mfhi mflo`	결과가 32비트를 넘으면 어디에 저장되는가?

QtSpim 준비

- **작성과 불러오기**: QtSpim에는 편집기가 없다. 외부 편집기로 .s 파일을 저장한 뒤 File → **Reinitialize and Load File**로 불러온다. 소스를 고칠 때마다 다시 불러와야 반영된다. 경로와 파일명은 영문으로 둔다.
- **실행**: **Single Step(F10)** 으로 한 줄씩 실행한다. 처음 몇 스텝은 QtSpim의 시작 코드이며, `jal main`을 지나면 내 코드가 시작된다.
- **레지스터 보기**: Int Regs 창에서 우클릭해 **Hex**로 둔다(실습 1에서는 Binary도 쓴다). HI, LO는 목록 위쪽 PC 근처에 있다.
- **Text 창 읽기**: 한 줄은 [주소] 기계어 실제 명령어 ; 줄번호: 내 소스 형식이다. 실제 명령어는 레지스터를 **번호**로, immediate를 **10진수**로 보여 준다.

모든 실습 코드는 아래 뼈대의 `main:` 아래에 넣는다. `main` 레이블이 없으면 실행할 때 오류가 난다.

```
        .text
        .globl main
main:
        # 여기에 실습 코드를 작성
        li   $v0, 10        # 프로그램 종료
        syscall
```



2. 실습 1 – 비트 연산 (Logical + Shift)

논리 명령어는 두 값을 **같은 자리 비트끼리** 연산하고, 시프트는 비트 전체를 옆으로 민다. 둘 다 값을 "수"가 아니라 "비트 무늬"로 다루는 명령어다.

핵심 개념

논리 명령어는 한쪽 피연산자를 **마스크**로 보면 쉽다. 시프트 세 명령어의 차이는 **빈자리를 무엇으로 채우는가** 하나뿐이다.

명령어	마스크 비트 0	마스크 비트 1	용도
and	0으로 지움	그대로	원하는 비트만 남기기(추출)
or	그대로	1로 세움	비트 세우기, 두 필드 합치기
xor	그대로	반전	비트 토글, 같은지 비교(같으면 0)
nor	–	–	nor rd, rs, $0 = NOT (MIPS에는 NOT이 없다)

명령어	방향	빈자리 채움	산술적 의미
sll	왼쪽	0	$\times 2^n$
srl	오른쪽	0	부호 없는 수 $\div 2^n$
sra	오른쪽	**원래 부호 비트**	부호 있는 수 $\div 2^n$

실습 코드

```
li   $t0, 0xF0F0F0F0
li   $t1, 0x00FF00FF
and  $t2, $t0, $t1      # 기대: 0x00F000F0
or   $t3, $t0, $t1      # 기대: 0xF0FFF0FF
xor  $t4, $t0, $t1      # 기대: 0xF00FF00F
nor  $t5, $t0, $t1      # 기대: 0x0F000F00  (or 결과의 반전)

li   $t6, 0x80000001
sll  $t7, $t6, 1        # 기대: 0x00000002
srl  $s0, $t6, 1        # 기대: 0x40000000
sra  $s1, $t6, 1        # 기대: 0xC0000000  (부호 유지)
```



기대값 구하는 법

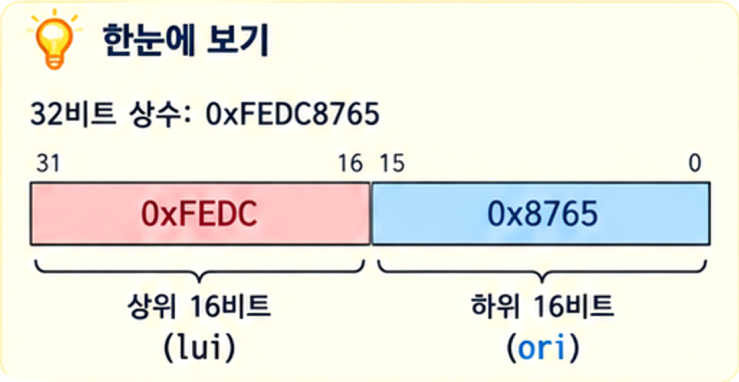
논리 연산은 **바이트 단위**로 계산한다. $t0의 바이트는 모두 F0이고 $t1의 바이트는 00 아니면 FF다. $t1이 00인 바이트에서는 and가 지우고(00) or/xor는 그대로 두며(F0), FF인 바이트에서는 and가 그대로 두고(F0) or가 채우고(FF) xor가 뒤집는다(0F).

시프트의 0x80000001은 **양 끝 비트(bit 31, bit 0)만** 1인 시험값이다.

```
원래  : 1000 0000 0000 0000 0000 0000 0000 0001   ; 0x80000001  (-2,147,483,647)
sll 1 : 0000 0000 0000 0000 0000 0000 0000 0010   ; 0x00000002  bit 31이 잘림 -> 오버플로
srl 1 : 0100 0000 0000 0000 0000 0000 0000 0000   ; 0x40000000  음수가 양수로 바뀜
sra 1 : 1100 0000 0000 0000 0000 0000 0000 0000   ; 0xC0000000  (-1,073,741,824) 부호 유지
```

3. 실습 2 – 32비트 상수를 레지스터로

MIPS 명령어는 32비트 고정 길이라 I-형식의 immediate에는 **16비트**만 남는다. 그래서 32비트 상수는 lui로 상위 16비트, ori로 하위 16비트를 나누어 넣는다. 지금까지 써 온 li는 어셈블러가 이 두 명령어로 바꿔 주는 의사 명령어다.



실습 코드

```
addi $s0, $0, 0x4f3c      # 16비트에 들어가는 상수: $s0 = 0x00004F3C

lui  $s0, 0xFEDC          # $s0 = 0xFEDC0000  (상위 절반 적재, 하위는 0)
ori  $s0, $s0, 0x8765     # $s0 = 0xFEDC8765  (하위 절반을 OR로 채움)
```

진행 순서

1. 한 줄씩 실행하며 lui 직후 $s0(R16)이 0xFEDC0000인 것을 반드시 확인한다.
2. $s0과 $s1(R17)을 비교해 어느 자리가 다른지 적는다.
3. 실습 1 코드를 불러온 Text 창에서 li $t0, 0xF0F0F0F0이 바뀐 두 명령어와 기계어를 찾아 적는다.



4. 실습 3 – 곱셈과 나눗셈

n비트 × n비트의 결과는 최대 2n비트다(2비트 11 × 11 = 4비트 1001). 32비트 곱은 최대 64비트이므로 전용 레지스터 **HI**(상위 32비트)와 **LO**(하위 32비트)에 나누어 담는다. 나눗셈도 몫과 나머지 두 결과가 나오므로 같은 레지스터를 쓴다.

명령어	동작	결과 위치
`mult $t2, $t3`	부호 있는 64비트 곱	HI = 상위 32비트, LO = 하위 32비트
`div $t2, $t3`	부호 있는 나눗셈	**LO = 몫, HI = 나머지**
`mfhi $t1` / `mflo $t1`	move from HI / LO	`$t1`
`mul $t1, $t2, $t3`	곱의 **하위 32비트만**	`$t1`
`div $t1, $t2, $t3`	몫을 바로 저장(의사 명령어)	`$t1` (= div + mflo)

mult와 2-피연산자 div에는 목적지 레지스터가 없다. 결과는 항상 HI/LO로 가며, 다음 mult/div에서 덮어써진다.

실습 코드

```
li   $t2, 100000
li   $t3, 100000          # 100000 x 100000 = 10,000,000,000 = 0x2_540BE400
mul  $t1, $t2, $t3        # 기대: $t1 = 0x540BE400  (앞의 2가 잘린 틀린 값)
mult $t2, $t3             # 기대: HI = 0x00000002, LO = 0x540BE400
mfhi $s0
mflo $s1

li   $t2, 17
li   $t3, 5
div  $t2, $t3             # 기대: LO = 3 (몫), HI = 2 (나머지)
mflo $t4                  # a / b 몫
mfhi $t5                  # a % b 나머지
```

결과 해석

- **mul은 결과가 32비트에 들어갈 때만 옳다.** 10,000,000,000은 34비트가 필요해서 mul은 하위 32비트(1,410,065,408)만 돌려준다. mult는 잘린 부분을 HI에 보존한다.

1 100000 × 100000 = 10,000,000,000 (= 0x2_540BE400)

방법	결과 (16진수)	설명
`mul $t1, $t2, $t3`	`$t1` = 0x540BE400	하위 32비트만 저장 (앞의 2가 잘린 값)
`mult $t2, $t3`	HI = 0x00000002	상위 32비트
	LO = 0x540BE400	하위 32비트

10,000,000,000 (64비트) = 0x000000002_540BE400

0000 0000 0000 0000 0000 0000 0000 0010	0101 0100 0000 1011 1110 0100 0000 0000
HI (상위 32비트)	LO (하위 32비트)

2 17 ÷ 5

레지스터	값 (10진수)	설명
LO	3	몫 (17 / 5)
HI	2	나머지 (17 % 5)

진행 순서

1. 실행 전에 10,000,000,000을 16진수로 바꿔 HI, LO를 예측해 기록표에 적는다.
2. mul 직후 $t1(R9)과 mult 직후 HI, LO를 비교한다.
3. div 직후 LO, HI를 확인하고 mflo/mfhi로 꺼낸다.



확인 문제

1. $t0 = 0xF0000000을 4비트 오른쪽으로 밀 때 srl과 sra의 결과를 각각 구하라.
3. $t2 = $t3 = 0x00010000일 때 mult 후의 HI, LO와 mul 후의 $t1은?

컴퓨터구조 · LAB-04
Quiz

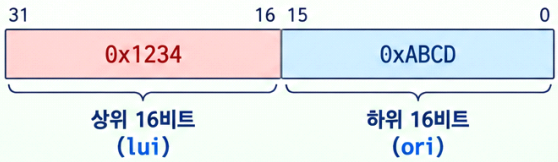
Q. lui + ori 대신 lui + addi 를 사용하면 안되는 이유는 무엇일까요?

MIPS에서 32비트 상수를 만들 때, 보통 상위 16비트는 lui로, 하위 16비트는 ori로 채웁니다.
그런데 하위 16비트를 채울 때 ori 대신 addi를 사용하면 안 되는 이유가 있을까요?

아래 예시를 보면서 어떤 문제가 생길지 생각해 봅시다!

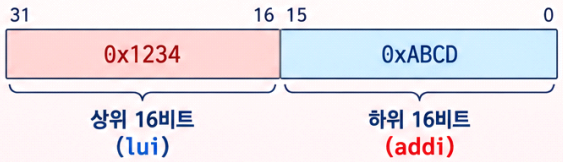
일반적으로 사용하는 방법
lui + ori

```
lui   $t0, 0x1234      # 상위 16비트 적재
ori   $t0, $t0, 0xABCD # 하위 16비트 채움
```



대신 이렇게 하면 어떨까요?
lui + addi

```
lui   $t0, 0x1234      # 상위 16비트 적재
addi  $t0, $t0, 0xABCD # 하위 16비트 채우기?
```





진지하게 고민해보세요!
시험에 나올수도 있습니다!

생각해 볼 포인트

- [ ] addi의 immediate는 어떤 값인가?
- [ ] 하위 16비트가 특정 값(예: 0x8000 이상)일 때는?
- [ ] lui로 적재된 상위 16비트에 어떤 영향을 줄까?
- [ ] ori와 addi의 연산 방식은 어떻게 다른가?

THINK
ANALYZE
SOLVE